



Complemento práctico: Gestión de Estado Global con Static

Del concepto teórico a la práctica: El algoritmo de "Auto-Incremento"

En la teoría vimos que `static` significa "perteneciente a la clase". Pero, ¿cómo se usa esto en el mundo real? En el video de esta actividad, analizamos un patrón de diseño fundamental en sistemas de gestión: la generación automática de IDs únicos.

Imagina una cola de impresión de Tickets. ¿Cómo sabe el ticket actual qué número le corresponde? ¿Cómo evitamos que dos tickets tengan el número 10? La respuesta no está en los objetos, sino en la clase misma.

En este documento desglosamos el ejercicio del "Ticket" paso a paso para entender cómo controlar la memoria compartida en Java.

Contenido Exclusivo:

El Desafío

Objetos aislados vs. Datos compartidos. Entenderás por qué la lógica normal de variables de instancia no sirve para numeración secuencial.

Caso de Estudio

Anatomía de la clase Ticket. Desglosaremos cada línea del código maestro para comprender el patrón completo.

Traza de Memoria

Qué ocurre en la RAM cuando hacemos `new`. Visualizarás el ciclo de vida de las variables estáticas.

Métodos de Clase

Cuándo una función no necesita un dueño. Aprenderás a crear utilidades globales sin instanciar objetos.

¿Por qué falla la lógica normal?

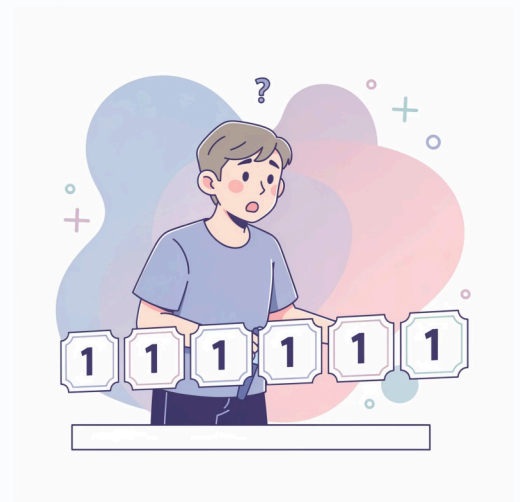
El Problema de los Objetos Aislados

Imagina que queremos numerar tickets: 1, 2, 3... Si usamos variables normales (de instancia), cada objeto nace **"amnésico"**. No sabe cuántos tickets se crearon antes que él.

El error común del estudiante:

```
public class Ticket {
    private int numero = 0; // Variable de
    instancia

    public Ticket() {
        this.numero++; // Siempre será 0 + 1 =
1
    }
}
```



Resultado: Todos los tickets del mundo tendrán el número 1.

La Solución

Necesitamos una variable que sobreviva a la creación de los objetos. Una variable que sea como una **"pizarra en la pared"** que todos los tickets puedan ver y modificar. Esa es la variable `static`.

Anatomía del Caso "Ticket"

Desglosando el Código del Video

El ejercicio del video propone una solución elegante usando dos tipos de variables complementarias. Esta distinción es **fundamental** para entender cómo funciona la memoria compartida en Java.



La Variable Global (La Pizarra)

```
private static int contador = 0;
```

- Al ser estática, solo existe UNA copia en toda la aplicación
- Pertenece a la clase Ticket, no al objeto t1 o t2
- Su trabajo es recordar cuál fue el último número emitido



La Variable Individual (El DNI)

```
private int numero;
```

- No es estática. Cada objeto tiene la suya
- Su trabajo es guardar el número que le tocó a este ticket específico para siempre
- Una vez asignado, nunca cambia



Analogía Clave: El contador static es como un dispensador de números en un banco: solo hay uno, todos lo usan, y cada persona se lleva su número único que ya no cambia.

El Algoritmo de Auto-Incremento

La Magia ocurre en el Constructor

El constructor es el momento perfecto para sincronizar lo global con lo individual. Este patrón es usado en miles de aplicaciones reales: bases de datos, sistemas de facturación, gestores de tareas, y más.

Código Maestro:

```
public class Ticket {  
    private static int contador = 0; // Memoria Global  
    private int numero;             // Memoria Individual  
  
    public Ticket(String comprador) {  
        // PASO 1: Actualizar la historia global  
        contador++;  
        // PASO 2: "Congelar" ese valor para mí  
        this.numero = contador;  
        System.out.println("Se generó el ticket nro: " + this.numero);  
    }  
}
```

Análisis del Flujo:

01	02	03
Nace Ticket A contador sube a 1 → TicketA.numero vale 1	Nace Ticket B contador sube a 2 → TicketB.numero vale 2	Ticket A no cambia Ticket A sigue valiendo 1, porque su variable numero no es estática. Es su identidad permanente.

Traza de Memoria: Ciclo de Vida de lo Estático

Es vital entender **cuándo nace cada cosa** para evitar NullPointerExceptions o errores lógicos. La secuencia temporal de inicialización es diferente para variables estáticas e de instancia.

Carga de la Clase (Class Loading)

Antes de que exista ningún objeto, Java lee la clase Ticket.

En ese instante, nace static int contador = 0. Ya ocupa memoria.

Esto ocurre una sola vez en todo el ciclo de vida del programa.

Instanciación (new)

Recién aquí nacen las variables de instancia (numero, fecha).

El objeto accede a la zona estática, modifica el contador y copia el valor.

Esto ocurre cada vez que hacemos new Ticket().



Conclusión Clave

Lo estático vive más tiempo que los objetos. Nace al principio del programa y muere cuando se cierra la aplicación. Es memoria persistente durante toda la ejecución.



Métodos Estáticos: Herramientas Utilitarias



El apunte teórico menciona la clase Utilidad. En el video, esto se traduce a métodos que consultan el estado global sin necesitar un ticket específico.

Ejemplo Práctico

Queremos saber cuántos tickets se vendieron hoy, pero no tenemos ningún ticket a mano.

```
// Método static: Se invoca desde la CLASE
public static int getTotalVendidos() {
    return contador;
}
```

Uso:

```
int total = Ticket.getTotalVendidos();
```

La Diferencia Clave

No hace falta hacer `new Ticket()` para preguntar esto. Es una pregunta a la **fábrica**, no al **producto**. Los métodos static son herramientas de clase, no de instancia.



Importante: Un método static NO puede acceder a variables de instancia (como `numero`), solo a otras variables static (como `contador`). ¿Por qué? Porque no tiene un objeto "dueño" que posea esas variables.

Ejemplo Complementario: El Banco

Para reforzar el concepto, veamos otro caso práctico que aparece frecuentemente en sistemas financieros reales. Este ejemplo muestra cuándo usar static y cuándo evitarlo.

Código de Ejemplo:

```
public class Cuenta {  
    double saldo;    // Cada uno tiene el suyo  
    static double interes; // Compartido  
}
```

// En el main:

```
Cuenta c1 = new Cuenta();
```

```
Cuenta c2 = new Cuenta();
```

```
c1.interes = 5.0;
```

// PREGUNTA DE EXAMEN:

```
System.out.println(c2.interes);
```

// RESPUESTA: 5.0 ✓

¿Por qué c2.interes vale 5.0?

Porque la variable es **static**, lo que significa que es **compartida por todas las instancias**. Cuando modificamos el interés a través de c1, estamos modificando la única copia que existe en memoria, por lo tanto c2 ve el mismo valor.

El saldo debe ser de instancia (cada cuenta tiene el suyo), pero el tipo de interés es static (el banco lo define para todas las cuentas). Esta es la lógica del negocio traducida a código.

Patrones de Uso: ¿Cuándo usar Static?



Usa Static Para:

- **Contadores globales:** IDs autoincrementales, estadísticas de uso
- **Constantes compartidas:** Tasas, límites, configuraciones
- **Métodos utilitarios:** Funciones matemáticas, conversiones
- **Puntos de acceso únicos:** Patrones Singleton

✓ Buen Uso de Static

- Generar IDs únicos (como Ticket)
- Almacenar constantes (Math.PI)
- Crear utilidades (Integer.parseInt())
- Contar instancias totales creadas

✗ Mal Uso de Static

- Datos que deben ser diferentes por objeto
- Información que cambia independientemente
- Abusar de static como variables globales
- Usar static para evitar crear objetos

Resumen de POO Intermedio

Conclusión de la Unidad 4

Con esta actividad cerramos la Unidad 4. Hemos aprendido a dominar las herramientas fundamentales de la programación orientada a objetos en Java. Estos conceptos son la base sólida sobre la que construiremos conocimientos más avanzados.



Identidad

Usar `this` para referirnos a nosotros mismos. Aprendimos a distinguir entre variables locales, parámetros y atributos del objeto.



Construcción

Usar sobrecarga de constructores para dar flexibilidad al nacimiento de objetos. Múltiples formas de crear instancias según necesidades.



Diagnóstico

Usar `toString()` para ver dentro de los objetos. Hacer que nuestros objetos puedan "hablar" y describirse a sí mismos.



Memoria Compartida

Usar `static` para conectar objetos entre sí mediante datos globales. El patrón de auto-incremento es solo el principio.



Próximo Paso

Estas herramientas son la base para la siguiente unidad, donde aprenderemos a relacionar clases distintas (Herencia y Polimorfismo). ¡Prepárate para el Trabajo Práctico Integrador!

